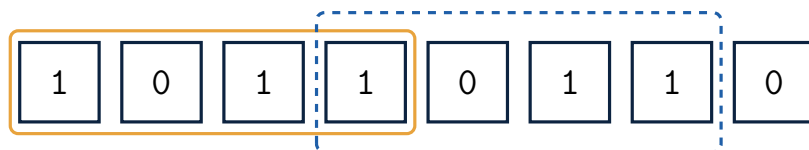


Lab 8

Sequence Detector FSMs

Moore & Mealy -- Overlap & Non-Overlap

Detecting the pattern 1011 in a serial bit stream:
four correct hardware designs, four different engineering trade-offs.



Cover illustration – IN = 10110110, two overlapping 1011 matches highlighted

Learning Objectives

Learning Objectives

By the end of this lab, you will be able to:

- Derive an overlap-aware FSM transition table for a fixed bit pattern using the failure-function method
- Explain, with a concrete transition, exactly what changes between an overlapping and a non-overlapping sequence detector
- Implement both Mealy-type (output on transition) and Moore-type (output on state) FSMs in synthesizable Verilog
- Simulate all four designs against a shared test vector and correctly predict/verify the number and timing of detected matches

Background

Background

Fixed bit-pattern detection in a live serial stream is not a textbook toy problem – it is the exact hardware task behind telecom frame synchronization. The DS1/T1 superframe standard locates frame boundaries using the 6-bit sync pattern 001011 (and its cyclic rotations); SONET uses the framing byte 11110110. Any hardware that must find such a pattern – possibly with overlapping occurrences – needs exactly the kind of FSM you will build in this lab.

In this lab the target pattern is 1011. Two independent design decisions produce four legitimate, distinct hardware implementations:

1. **Mealy vs. Moore** – does the detected-match signal come out combinational on the transition (Mealy, same cycle) or only once the FSM settles into a dedicated “matched” state (Moore, one cycle later)?
2. **Overlap vs. non-overlap** – after a match, may the trailing bits of that match be reused as the start of the next candidate match, or must the search restart from scratch?

Shared Test Stream (use this for all four questions)

IN = 1 0 1 1 0 1 1 0 (one bit per rising clock edge, bit 1 applied first)

Overlapping detectors must report 2 matches: bits 1–4, and bits 4–7 (bit 4 is reused).

Non-overlapping detectors must report 1 match: bits 1–4 only.

1 Question 1 -- Mealy Machine, Overlap Allowed

Q1: Mealy, overlapping detector

Design and simulate a Mealy-type FSM that detects 1011 on serial input *in*, asserting *out* = 1 *combinatorially, in the same clock cycle* the 4th matching bit arrives. Overlapping matches must be permitted: a 1 that completes one match may simultaneously begin the next.

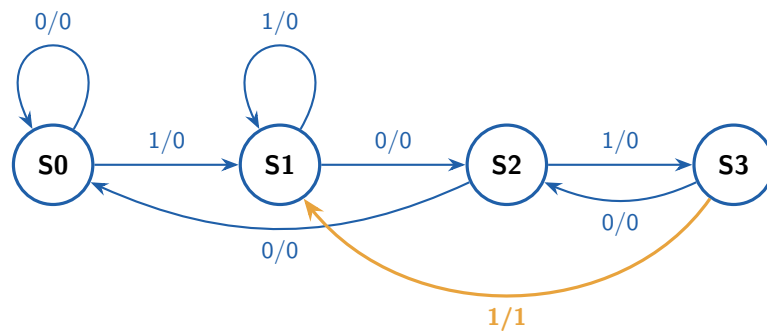


Figure 1 – Q1 state diagram: Mealy, overlap allowed (amber = the overlap-defining transition)

Verified transition table

State	in = 0	in = 1
S0	S0 / 0	S1 / 0
S1	S2 / 0	S1 / 0
S2	S0 / 0	S3 / 0
S3	S2 / 0	S1 / 1

```

1 module mealy_overlap (
2     input wire clk,
3     input wire rst,
4     input wire in,
5     output reg out
6 );
7     localparam S0=2'b00, S1=2'b01, S2=2'b10, S3=2'b11;
8     reg [1:0] state, next_state;
9
10    always @(posedge clk or posedge rst) begin
11        if (rst) state <= S0;
12        else state <= next_state;
13    end
14
15    always @(*) begin
16        next_state = S0;
17        out = 1'b0;
18        case (state)
19            S0: next_state = in ? S1 : S0;
20            S1: next_state = in ? S1 : S2;
21            S2: next_state = in ? S3 : S0;
22            S3: begin
23                if (in) begin

```

```

24         next_state = S1;    // overlap: reuse the matching '1'
25         out        = 1'b1;  // Mealy: asserted this same cycle
26     end else begin
27         next_state = S2;
28     end
29 end
30 endcase
31 end
32 endmodule

```

Listing 1: mealy_overlap.v

Verified Result

On the shared stream, out pulses on the 4th and 7th applied bits – 2 matches, both in the *same* cycle the triggering bit is applied.

2 Question 2 -- Mealy Machine, Non-Overlapping

Q2: Mealy, non-overlapping detector

Same Mealy structure as Q1, but on detecting a match, discard the matching tail entirely and restart the search from S0 – no bit may be reused to begin the next candidate match.

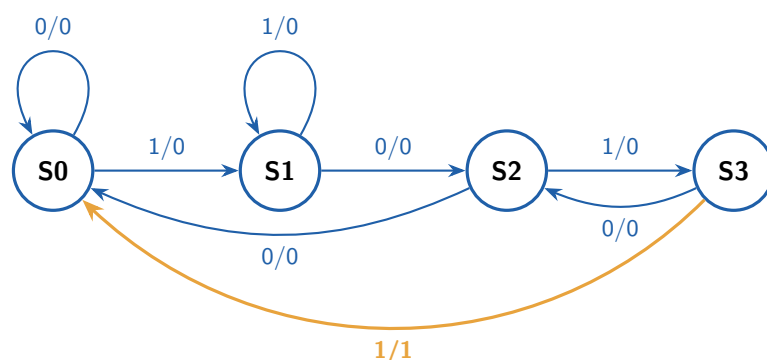


Figure 2 – Q2 state diagram: Mealy, no overlap (amber = full reset to S0 on match, the only edge that differs from Figure 1)

Verified transition table

State	in = 0	in = 1
S0	S0 / 0	S1 / 0
S1	S2 / 0	S1 / 0
S2	S0 / 0	S3 / 0
S3	S2 / 0	S0 / 1

```

1 module mealy_nooverlap (
2     input  wire clk,
3     input  wire rst,

```

```

4   input  wire in,
5   output reg  out
6 );
7   localparam S0=2'b00, S1=2'b01, S2=2'b10, S3=2'b11;
8   reg [1:0] state, next_state;
9
10  always @(posedge clk or posedge rst) begin
11      if (rst) state <= S0;
12      else     state <= next_state;
13  end
14
15  always @(*) begin
16      next_state = S0;
17      out        = 1'b0;
18      case (state)
19          S0: next_state = in ? S1 : S0;
20          S1: next_state = in ? S1 : S2;
21          S2: next_state = in ? S3 : S0;
22          S3: begin
23              if (in) begin
24                  next_state = S0; // full reset, no overlap reuse
25                  out        = 1'b1;
26              end else begin
27                  next_state = S2;
28              end
29          end
30      endcase
31  end
32 endmodule

```

Listing 2: mealy_nooverlap.v

Verified Result

On the shared stream, out pulses only on the 4th applied bit – 1 match. The FSM has fully reset by bit 5 and must re-climb $S0 \rightarrow S1 \rightarrow S2 \rightarrow S3$ before it can match again, so the bit-7 candidate is never detected.

3 Question 3 -- Moore Machine, Overlap Allowed

Q3: Moore, overlapping detector

Design a Moore-type FSM for the same overlap-allowed policy as Q1, where out is a pure function of the *current state*. This requires a 5th state $S4$ that is entered on a match; $S4$'s own outgoing transitions must mirror $S1$'s, since after outputting the match the FSM's position for future matching is equivalent to "just matched a single 1".

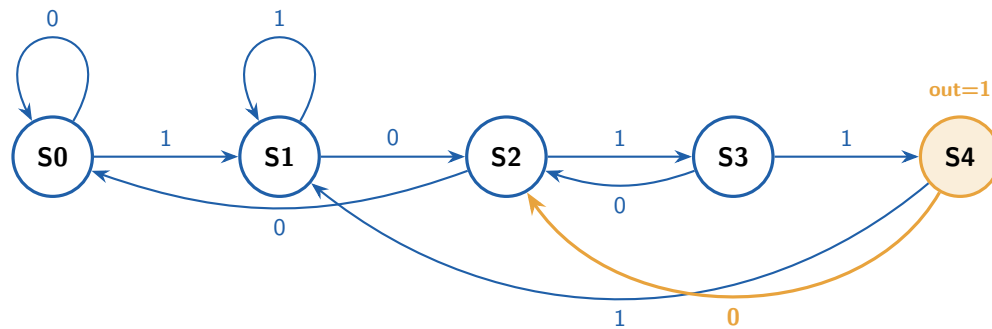


Figure 3 – Q3 state diagram: Moore, overlap allowed (S4 = the unique output=1 state; amber edge is the one that differs from Figure 4)

Verified transition table

State	in = 0	in = 1	Output
S0	S0	S1	0
S1	S2	S1	0
S2	S0	S3	0
S3	S2	S4	0
S4	S2	S1	1

```

1 module moore_overlap (
2     input wire clk,
3     input wire rst,
4     input wire in,
5     output wire out
6 );
7     localparam S0=3'd0, S1=3'd1, S2=3'd2, S3=3'd3, S4=3'd4;
8     reg [2:0] state, next_state;
9
10    always @(posedge clk or posedge rst) begin
11        if (rst) state <= S0;
12        else state <= next_state;
13    end
14
15    always @(*) begin
16        case (state)
17            S0: next_state = in ? S1 : S0;
18            S1: next_state = in ? S1 : S2;
19            S2: next_state = in ? S3 : S0;
20            S3: next_state = in ? S4 : S2;
21            S4: next_state = in ? S1 : S2; // mirrors S1's transitions
22            default: next_state = S0;
23        endcase
24    end
25
26    assign out = (state == S4); // pure function of state: Moore
27 endmodule

```

Listing 3: moore_overlap.v

Verified Result

Same 2 matches as Q1, but each out pulse appears *one clock cycle later* than its Q1 counterpart – registered on the state, not combinational on the transition.

4 Question 4 -- Moore Machine, Non-Overlapping**Q4: Moore, non-overlapping detector**

Same Moore structure as Q3, but S4's outgoing transitions now mirror S0's (full reset one cycle after the match) – the Moore parallel to Q2's non-overlap policy.

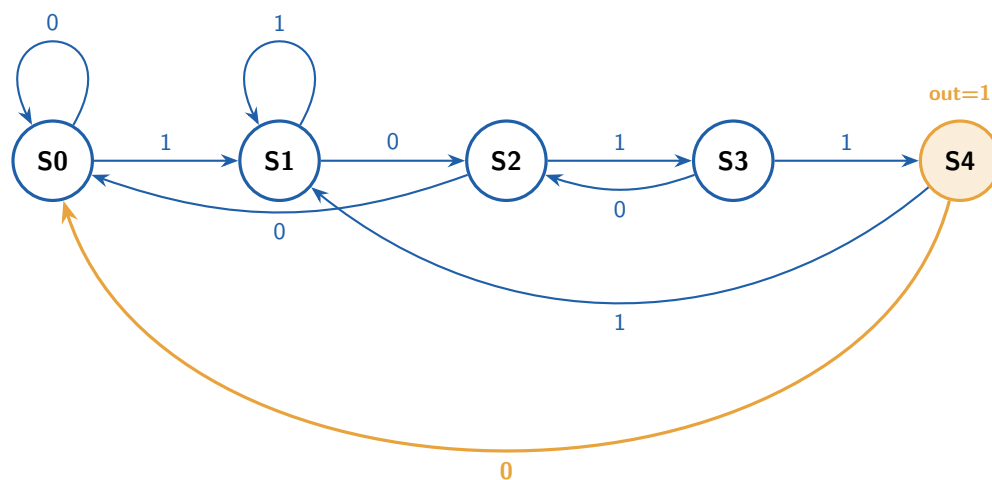


Figure 4 – Q4 state diagram: Moore, no overlap (amber edge: S4 falls all the way back to S0 instead of S2)

Verified transition table

State	in = 0	in = 1	Output
S0	S0	S1	0
S1	S2	S1	0
S2	S0	S3	0
S3	S2	S4	0
S4	S0	S1	1

```

1 module moore_nooverlap (
2     input  wire clk,
3     input  wire rst,
4     input  wire in,
5     output wire out
6 );
7     localparam S0=3'd0, S1=3'd1, S2=3'd2, S3=3'd3, S4=3'd4;
8     reg [2:0] state, next_state;
9 
```

```

10  always @(posedge clk or posedge rst) begin
11      if (rst) state <= S0;
12      else      state <= next_state;
13  end
14
15  always @(*) begin
16      case (state)
17          S0: next_state = in ? S1 : S0;
18          S1: next_state = in ? S1 : S2;
19          S2: next_state = in ? S3 : S0;
20          S3: next_state = in ? S4 : S2;
21          S4: next_state = in ? S1 : S0; // mirrors S0's transitions
22          default: next_state = S0;
23      endcase
24  end
25
26  assign out = (state == S4);
27 endmodule

```

Listing 4: moore_nooverlap.v

Verified Result

Only 1 match, one cycle after bit 4 – no second pulse near bit 7, for the same reason as Q2.

5 Testbench Template

Use the following structure for all four modules – instantiate the module under test and reuse this stimulus:

```

1  module tb_top;
2      reg clk = 0, rst = 1, in;
3      wire out;
4      integer i;
5      reg [7:0] stream = 8'b10110110; // bit 1 = MSB, applied first
6
7      // Instantiate the design under test, e.g.:
8      // mealy_overlap dut (.clk(clk), .rst(rst), .in(in), .out(out));
9
10     always #5 clk = ~clk;
11
12     initial begin
13         rst = 1; in = 0;
14         @(negedge clk); rst = 0;
15         for (i = 7; i >= 0; i = i - 1) begin
16             in = stream[i];
17             @(negedge clk); // sample 'out' just after the clock edge
18             $display("t=%0t bit=%b out=%b", $time, in, out);
19         end
20         $finish;
21     end
22 endmodule

```

Listing 5: Shared testbench structure

6 Summary Comparison

Build	Matches on shared stream	Detection timing
Q1: Mealy, overlap	2 (bit 4, bit 7)	Same cycle as 4th bit
Q2: Mealy, no overlap	1 (bit 4)	Same cycle as 4th bit
Q3: Moore, overlap	2 (bit 4, bit 7)	One cycle after each match
Q4: Moore, no overlap	1 (bit 4)	One cycle after match

Deliverables

Deliverables Checklist

- ☐ Four synthesizable modules (Q1–Q4), matching the verified transition tables above
- ☐ Four testbenches driving the shared stream 10110110
- ☐ One waveform screenshot per module, annotated with which bit(s) triggered out = 1
- ☐ One-paragraph write-up: for each of the two design axes (Mealy/Moore, overlap/no-overlap), state in your own words the single structural change in the FSM that causes the behavioral difference observed in simulation